

光量子测控系统通信协议文档

本源量子

2026 年 2 月 10 日

目录

1 通信协议	3
2 消息定义 (Router-Dealer 模式)	3
2.1 下发计算任务	3
2.2 任务状态查询	4
2.3 返回计算结果	5
2.4 心跳消息	6
2.5 获取芯片校准时间	7
2.6 获取 RB 实验数据	8
2.7 获取芯片配置参数	10
2.8 请求指定任务结果	11
2.9 设置独占机时	12
2.10 释放独占机时	12
3 订阅消息 (PUB-SUB 模式)	13
3.1 任务状态	13
3.2 芯片配置信息更新标志	14
3.3 芯片资源状态	14
3.4 自动校准开始信息	16
3.5 自动校准结束信息	17
3.6 芯片维护信息	17
4 类型定义	18
4.1 任务状态类型	18
4.2 错误码类型	18

5 指令定义	19
5.1 包含的基础门类型与参数说明	19

1 通信协议

ZeroMQ+Json 消息体。

ZeroMQ 是一个成熟高效的第三方通信库，可方便的实现 C++ 和 Python 程序之间的高效通信。

ZeroMQ 有多种工作模式，目前司南和测控系统对接类似远端调用程序，这里使用“请求回应模型”。

2 消息定义 (Router-Dealer 模式)

2.1 下发计算任务

功能：该消息用于司南系统向测控系统下发计算任务。

参数定义：

- **MsgType (String):** [不可空] 消息类型，固定为 **MsgTask**。
- **SN (UInt32_t):** [不可空] 消息序列 ID。
- **TaskId (String):** [不可空] 任务 ID。
- **ConvertQProg (String):** [不可空] 测控要执行的指令（光量子平台专用格式）。
- **Configure.TaskPriority (UInt32_t):** [可空，默认为 0] 任务优先级。目前只支持 2 级优先级：0（普通任务），1（高优先级）。
- **Configure.Shot (UInt32_t):** [不可空] 重复次数。取值区间：100~10000，超出后默认 1000。
- **Configure.IsExperiment (Bool):** [可空] 是否实验模式。true：实验模式；false：非实验模式（普通用户任务）。该字段为空时，代表普通用户任务模式。
- **Configure.ClockCycle (UInt32_t):** [可空] 执行时序。最大线路执行周期，单位：微秒。司南根据测控返回的芯片最大执行周期自动决定是否传递该参数。比如测控返回当前芯片的普通执行周期是 100us，最大执行周期是 500us，如果一个任务的执行时序超过 100us，小于 500us，则根据具体线路时序情况，调整该参数；如果用户线路执行时序小于 100us，则不需要传递该参数。
- **Configure.PointLabel (UInt32_t):** [不可空] 应用工厂标签，暂定为 128。

返回信息：

- **MsgType (String):** [不可空] 消息类型，固定为 **MsgTaskAck**。
- **SN (UInt32_t):** [不可空] 消息序列 ID。
- **ErrCode (UInt32_t):** [不可空] 错误码。

- **ErrInfo (String):** [条件必填] 错误信息。无错误时可为空；发生错误时不可空。

请求示例:

```

1 {
2     "MsgType": "MsgTask",
3     "SN": 133,
4     "TaskId": "11D919FA044846F3B4DF453A827AE901",
5     "ConvertQProg": "[{\\"RZ\\": [0, 180.0]}, {\\"CNOT\\": [1, 2]}, {\\"H\\": [2]}, {\\"Measure\\": [[1, 0]]}]",
6     "Configure": {
7         "Shot": 1000,
8         "TaskPriority": 1,
9         "IsExperiment": true,
10        "PointLabel": 128
11    }
12 }
```

返回示例:

```

1 {
2     "MsgType": "MsgTaskAck",
3     "SN": 133,
4     "ErrCode": 2,
5     "ErrInfo": "configure error."
6 }
```

2.2 任务状态查询

功能: 司南向测控系统发送任务状态查看信息，用于确定当前任务计算状态。当司南长时间接收不到计算结果时，可通过此消息，向测控查询任务状态，防止异常情况下，司南死等。

参数定义:

- **MsgType (String):** [不可空] 消息类型，固定为 TaskStatus。
- **SN (Uint32_t):** [不可空] 消息序列 ID。
- **TaskId (String):** [不可空] 任务 ID。

返回信息:

- **MsgType (String):** [不可空] 消息类型，固定为 TaskStatusAck。
- **SN (Uint32_t):** [不可空] 消息序列 ID。

- **TaskId** (String): [不可空] 任务 ID。
- **TaskStatus** (Uint32_t): [不可空] 任务状态，参考“类型定义”章节。

请求示例:

```
1 {
2     "MsgType": "TaskStatus",
3     "SN": 10086,
4     "TaskId": "11D919FA044846F3B4DF453A827AE901"
5 }
```

返回示例:

```
1 {
2     "MsgType": "TaskStatusAck",
3     "SN": 10086,
4     "TaskId": "11D919FA044846F3B4DF453A827AE901",
5     "TaskStatus": 3
6 }
```

2.3 返回计算结果

功能: 测控系统向司南系统返回计算结果。

参数定义:

- **MsgType** (String): [不可空] 消息类型，固定为 **MsgTaskResult**。
- **SN** (Uint32_t): [不可空] 消息序列 ID。
- **TaskId** (String): [不可空] 任务 ID。
- **ProbCount** (Int 数组): [不可空] 坍塌次数，各个 Key 对应的坍塌次数。
- **NoteTime.CompileTime** (Int): [不可空] 编译耗时信息，单位 ms。
- **NoteTime.PendingTime** (Int): [不可空] 排队耗时信息，单位 ms。
- **NoteTime.MeasureTime** (Int): [不可空] 测量耗时信息，单位 ms。
- **NoteTime.PostProcessTime** (Int): [不可空] 后处理耗时信息，单位 ms。
- **ErrorCode** (Uint32_t): [不可空] 错误码。
- **ErrInfo** (String): [条件必填] 错误信息。

返回信息:

- **MsgType** (String): [不可空] 消息类型，固定为 **MsgTaskResultAck**。
- **SN** (Uint32_t): [不可空] 消息序列 ID。
- **ErrorCode** (Uint32_t): [不可空] 错误码。

- **ErrInfo (String):** [条件必填] 错误信息。

请求示例:

```

1 {
2     "MsgType": "MsgTaskResult",
3     "SN": 133,
4     "TaskId": "11D919FA044846F3B4DF453A827AE901",
5     "#key": "量子态统一用16进制字符串表示",
6     "Key": [["0x0", "0x1"], ["0x0", "0x1"]],
7     "ProbCount": [[111, 656], [103, 703]],
8     "NoteTime":
9     {
10         "CompileTime": 1,
11         "PendingTime": 94,
12         "MeasureTime": 2306,
13         "PostProcessTime": 105
14     },
15     "ErrCode": 0,
16     "ErrInfo": ""
17 }
```

返回示例:

```

1 {
2     "MsgType": "MsgTaskResultAck",
3     "SN": 133,
4     "ErrCode": 3,
5     "ErrInfo": "data error."
6 }
```

2.4 心跳消息

功能: 用户探测司南系统和测控系统的连接状态。由测控系统维护系统服务，司南系统主动发送心跳包，测控收到心跳包后，发送心跳回应。

参数定义:

- **MsgType (String):** [不可空] 消息类型，固定为 MsgHeartbeat。
- **SN (Uint32_t):** [不可空] 心跳消息序列 ID。
- **TimeStamp (Uint64_t):** [不可空] 64 位整形时间戳，毫秒。
- **ChipID (Uint32_t):** [不可空] 芯片名称，例如：72。

返回信息:

- **MsgType** (String): [不可空] 消息类型，固定为 **MsgHeartbeatAck**。
- **SN** (Uint32_t): [不可空] 心跳消息序列 ID。
- **backend** (Uint32_t): [不可空] 后端芯片 ID。
- **TimeStamp** (Uint64_t): [不可空] 64 位整形时间戳，毫秒。
- **Topic** (String): [不可空] 订阅消息主题。

请求示例:

```

1 {
2     "MsgType": "MsgHeartbeat",
3     "SN": 133,
4     "ChipID": 72,
5     "TimeStamp": 1638769359507
6 }
```

返回示例:

```

1 {
2     "MsgType": "MsgHeartbeatAck",
3     "SN": 133,
4     "backend": 72,
5     "TimeStamp": 1638769359517,
6     "Topic": "Y4-231011-设计验证-72bit_300pin_V9.2.3_Base-3#_|_Y4
7         -231011-设计验证-72bit_300pin_V9.2.3_Base-3#"
8 }
```

2.5 获取芯片校准时间

功能: 用户主动获取测控系统对芯片最后一次校准的时间。由测控系统返回。

参数定义:

- **MsgType** (String): [不可空] 消息类型，固定为 **GetUpdateTime**。
- **SN** (Uint32_t): [不可空] 消息序列 ID。

返回信息:

- **MsgType** (String): [不可空] 消息类型，固定为 **GetUpdateTimeAck**。
- **SN** (Uint32_t): [不可空] 消息序列 ID。
- **backend** (Uint32_t): [不可空] 后端芯片 ID。
- **LastUpdateTime.qubit** (Uint32_t): [不可空] 芯片校准时间戳的比特名称 (整型数据)。

- **LastUpdateTime.timeStamp** (Uint64_t): [不可空] 芯片校准时间戳的比特对应时间（64 位整型时间戳，毫秒）。
- **ErrCode** (Uint32_t): [不可空] 错误码。
- **ErrInfo** (String): [条件必填] 错误信息。

请求示例:

```
1 {
2     "MsgType": "GetUpdateTime",
3     "SN": 133
4 }
```

返回示例:

```
1 {
2     "MsgType": "GetUpdateTimeAck",
3     "SN": 133,
4     "backend": 72,
5     "LastUpdateTime":
6     {
7         "qubit": [43, 45, 46, 48, 49, 52, 53, 54, 60],
8         "timeStamp": [1687243363189, 1687243363189, 1687243363189,
9             1687243363189, 1687243363189,
10            1687243363189, 1687243363189,
11            1687243363189]
12     },
13     "ErrCode": 0,
14     "ErrInfo": ""
15 }
```

2.6 获取 RB 实验数据

功能: 用户主动获取 RB 实验数据。由测控系统返回。

参数定义:

- **MsgType** (String): [不可空] 消息类型，固定为 GetRBData。
- **SN** (Uint32_t): [不可空] 消息序列 ID。
- **ChipID** (Uint32_t): [不可空] 芯片名称，例如：72。

返回信息:

- **MsgType** (String): [不可空] 消息类型，固定为 GetRBDataAck。
- **SN** (Uint32_t): [不可空] 消息序列 ID。

- **backend** (Uint32_t): [不可空] 后端芯片 ID。
- **SingleGateCircuitDepth** (Uint32_t): [不可空] RB 实验单门线路最大深度。
- **DoubleGateCircuitDepth** (Uint32_t): [不可空] RB 实验双门线路最大深度。
- **SingleGateFidelity** (String): [不可空] 单比特保真度 (字符串类型 map 表)。
- **DoubleGateFidelity** (String): [不可空] 双比特保真度 (字符串类型 map 表)。
- **ErrCode** (Uint32_t): [不可空] 错误码。
- **ErrInfo** (String): [条件必填] 错误信息。

请求示例:

```
1 {
2     "MsgType": "GetRBData",
3     "SN": 133,
4     "ChipID": 72
5 }
```

返回示例:

```
1 {
2     "MsgType": "GetRBDataAck",
3     "SN": 133,
4     "backend": 72,
5     "SingleGateCircuitDepth": [50,50,50,50,50,50,50,50,50,50],
6     "DoubleGateCircuitDepth": [0,50,50,50,50,50,50,0],
7     "SingleGateFidelity":
8     {
9         "qubit": ["45","46","48","52"],
10        "fidelity": [0.9,0.9,0.9,0.9]
11    },
12    "DoubleGateFidelity":
13    {
14        "qubitPair": ["45-46","46-52","48-54","52-53"],
15        "fidelity": [0.9,0.9,0.9,0.9]
16    },
17    "ErrCode": 0,
18    "ErrInfo": ""
19 }
```

2.7 获取芯片配置参数

功能：获取芯片配置参数 (json 数据)，由测控返回。

参数定义：

- **MsgType** (String): [不可空] 消息类型，固定为 GetChipConfig。
- **SN** (Uint32_t): [不可空] 消息序列 ID。
- **ChipID** (Uint32_t): [不可空] 芯片名称，例如：72。

返回信息：

- **MsgType** (String): [不可空] 消息类型，固定为 GetChipConfigAck。
- **SN** (Uint32_t): [不可空] 消息序列 ID。
- **backend** (Uint32_t): [不可空] 后端芯片 ID。
- **PointLabelList** (Uint32_t 数组): [不可空] 当前芯片支持的工作区模式列表。
- **ChipConfig** (String): [不可空] 芯片参数 json 字符串。
- **ErrCode** (Uint32_t): [不可空] 错误码。
- **ErrInfo** (String): [条件必填] 错误信息。

请求示例：

```
1 {  
2     "MsgType": "GetChipConfig",  
3     "SN": 133,  
4     "ChipID": 72  
5 }
```

返回示例：

```
1 {  
2     "MsgType": "GetChipConfigAck",  
3     "SN": 133,  
4     "backend": 72,  
5     "PointLabelList": [1, 2],  
6     "ChipConfig": {  
7         "1": "QuantumChipArch.json 相关配置信息",  
8         "2": "QuantumChipArch.json 相关配置信息"  
9     },  
10    "ErrCode": 0,  
11    "ErrInfo": ""  
12 }
```

2.8 请求指定任务结果

功能：用于在异常情况下，司南未正确接收到测控返回的计算结果时，司南主动向测控请求指定任务的计算结果信息。

参数定义：

- **MsgType (String):** [不可空] 消息类型，固定为 `GetTaskResult`。
- **SN (Uint32_t):** [不可空] 消息序列 ID。
- **TaskId (String):** [不可空] 任务 ID。

返回信息：返回结果信息和“2.3 返回计算结果”章节中格式一致，但删除“TaskResult”和“FidelityMat”字段信息。**注意：**返回结果中的消息流水号和获取结果消息的流水号保持一致。

请求示例：

```
1 {
2     "MsgType": "GetTaskResult",
3     "SN": 134,
4     "TaskId": "11D919FA044846F3B4DF453A827AE901"
5 }
```

返回示例：

```
1 {
2     "MsgType": "MsgTaskResult",
3     "SN": 134,
4     "TaskId": "11D919FA044846F3B4DF453A827AE901",
5     "#key": "量子态统一用16进制字符串表示",
6     "Key": [["0x0", "0x1"], ["0x0", "0x1"]],
7     "ProbCount": [[111, 656], [103, 703]],
8     "NoteTime":
9     {
10         "CompileTime": 1,
11         "PendingTime": 94,
12         "MeasureTime": 2306,
13         "PostProcessTime": 105
14     },
15     "ErrCode": 0,
16     "ErrInfo": ""
17 }
```

2.9 设置独占机时

功能：用于在特定需求场景下，司南设定独占机时，确保在独占机时内任务无需排队。

参数定义：

- **MsgType** (String): [不可空] 消息类型，固定为 **SetVip**。
- **SN** (Uint32_t): [不可空] 消息序列 ID。
- **OffsetTime** (Uint64_t): [不可空] 相对系统当前时间的偏移时间（单位：秒）。
- **ExclusiveTime** (Uint64_t): [不可空] 独占机时时长（单位：秒）。

返回信息：

- **MsgType** (String): [不可空] 消息类型，固定为 **SetVipAck**。
- **SN** (Uint32_t): [不可空] 消息序列 ID。
- **ErrCode** (Uint32_t): [不可空] 错误码。
- **ErrInfo** (String): [条件必填] 错误信息。

请求示例：

```
1 {  
2     "MsgType": "SetVip",  
3     "SN": 134,  
4     "OffsetTime": 120,  
5     "ExclusiveTime": 600  
6 }
```

返回示例：

```
1 {  
2     "MsgType": "SetVipAck",  
3     "SN": 134,  
4     "ErrCode": 0,  
5     "ErrInfo": ""  
6 }
```

2.10 释放独占机时

功能：用于在完成独占机时任务计算后，释放独占机时操作，恢复普通模式。

参数定义：

- **MsgType** (String): [不可空] 消息类型，固定为 **ReleaseVip**。
- **SN** (Uint32_t): [不可空] 消息序列 ID。

返回信息：

- **MsgType** (String): [不可空] 消息类型, 固定为 `ReleaseVipAck`。
- **SN** (Uint32_t): [不可空] 消息序列 ID。
- **ErrCode** (Uint32_t): [不可空] 错误码。
- **ErrInfo** (String): [条件必填] 错误信息。

请求示例:

```
1 {
2     "MsgType": "ReleaseVip",
3     "SN": 135
4 }
```

返回示例:

```
1 {
2     "MsgType": "ReleaseVipAck",
3     "SN": 135,
4     "ErrCode": 0,
5     "ErrInfo": ""
6 }
```

3 订阅消息 (PUB-SUB 模式)

3.1 任务状态

功能: 在任务计算的阶段变更时, 由测控推送任务实时状态信息。

Operation: 固定为 `bytes: task_status`。

推送消息格式:

- **MsgType** (String): [不可空] 消息类型, 固定为 `TaskStatus`。
- **SN** (Uint32_t): [不可空] 消息序列 ID。
- **TaskId** (String): [不可空] 任务 ID。
- **TaskStatus** (Uint32_t): [不可空] 任务状态, 参考任务状态定义。

示例:

```
1 {
2     "MsgType": "TaskStatus",
3     "SN": 0,
4     "TaskId": "11D919FA044846F3B4DF453A827AE901",
5     "TaskStatus": 3
6 }
```

3.2 芯片配置信息更新标志

功能：在芯片参数变更后，测控一体机推送消息，告知司南可以通过 Router-Dealer 模式发送获取请求查询最新芯片配置信息。

Operation：固定为 bytes: chip_update。

推送消息格式：

- UpdateFlag (Bool): [不可空] 更新标识。
- LastUpdateTime (Uint64_t): [不可空] 最后更新时间。

示例：

```
1 {  
2   "UpdateFlag": true,  
3   "LastUpdateTime": 1705307288685  
4 }
```

3.3 芯片资源状态

功能：在测控一体机计算任务时，在多线程模式下，只要芯片资源发生了改变（锁住、释放），测控服务就会推送线程详细数据至司南（各个测控服务线程的 qubit 使用情况）；QPilot 可以只关注每个线程（t0, t1, t2...）的 use_bits 字段。

Operation：固定为 bytes: probe。

推送消息格式：

- core_thread.t0 (Object): [不可空] 测控服务的线程名。
- core_thread.t0.use_bits (String 数组): [不可空] 当前线程所有比特。

示例：

```
1 {  
2   "inst_status": 1,  
3   "linked": 1,  
4   "timestamp": 1695182448.7963016,  
5   "scheduler": { "status": "InitialState", "queue_len": 3 },  
6   "core_status": { "empty_thread": 3, "pause_read": 0, "thread_num  
7     ": 5 },  
8   "core_thread": {  
9     "t0": {  
10       "status": "waiting",  
11       "thread_id": "t0",  
12       "task_id": "650a6e6f197b42e9aeaad6d6",  
13       "start_time": 1695182447.5371444,  
        "user": "admin3",
```

```

14         "env_bits": [
15             "<c45-46||0>",
16             "<q62||0>",
17             "<c40-46||0>",
18             "<q59||1>"],
19     "use_bits": [
20         "q56",
21         "q18",
22         "q68",
23         "q69",
24         "q6",
25         "q2",
26         "q67"]
27 },
28 "t1": {
29     "status": "ready",
30     "task_id": null,
31     "start_time": null,
32     "user": null,
33     "env_bits": [],
34     "use_bits": []
35 },
36 "t2": {
37     "status": "ready",
38     "task_id": null,
39     "start_time": null,
40     "user": null,
41     "env_bits": [],
42     "use_bits": []
43 },
44 "t3": {
45     "status": "waiting",
46     "thread_id": "t3",
47     "task_id": "650a6e6e197b42e9aeaad6d0",
48     "start_time": 1695182446.8370538,
49     "user": "admin3",
50     "env_bits": [
51         "<q56||1>",
52         "<c45-46||2>",
53         "<q62||2>",

```

```

54         "<c40-46||2>",
55         "<c56-57||2>",
56         "<c43-44||2>",
57         "<c44-45||2>"
58     ],
59     "use_bits": [
60         "q59",
61         "q30",
62         "q50",
63         "q33",
64         "q47",
65         "q19",
66         "q38"
67     ]
68 },
69 "t4": {
70     "status": "ready",
71     "task_id": null,
72     "start_time": null,
73     "user": null,
74     "env_bits": [],
75     "use_bits": []
76 }
77 }
78 }

```

3.4 自动校准开始信息

功能：在测控一体机进行自动校准时，会推送校准比特信息至司南，此时间段内校准中比特不可用。

Operation：固定为 bytes: calibration_start。

推送消息格式：

- qubits (String 数组): [不可空] 校准后的单比特信息。
- couplers (String 数组): [不可空] 耦合器。
- pairs (String 数组): [不可空] 校准后的比特对信息。
- discriminators (String 数组): [不可空] 鉴别器。
- point_label (Int): [不可空] 用于表示当前校准作用于哪个标签。

示例：


```

1 {
2     "config_flag": false,
3     "qubits":["q0", "q1", "q2", "q3"],
4     "couplers":["c0-1", "c11-12", "c2-3"],
5     "pairs": ["q0q1", "q2q3"],
6     "discriminators":["q0_01.bin", "q1_01.bin", "q2_01.bin", "q3_01.
7         bin"],
8     "point_label": 2
9 }

```

3.5 自动校准结束信息

功能：完成校准后会推送完成消息至司南，比特恢复可用。

Operation：固定为 bytes: calibration_done。

推送消息格式：

- **qubits** (String 数组): [不可空] 校准后的单比特信息。
- **couplers** (String 数组): [不可空] 耦合器。
- **pairs** (String 数组): [不可空] 校准后的比特对信息。
- **discriminators** (String 数组): [不可空] 鉴别器。

示例：

```

1 {
2     "qubits":["q0", "q1", "q2", "q3"],
3     "couplers":["c0-1", "c11-12", "c2-3"],
4     "pairs": ["q0q1", "q2q3"],
5     "discriminators":["q0", "q1", "q2", "q3"],
6     "config_flag":true,
7     "point_label": 2
8 }

```

3.6 芯片维护信息

功能：接收芯片维护开始和结束信息，实现司南自动暂停。如果维护时间超过 2 个小时，则司南向云平台推送维护下线通知；如果维护时间不超过 2 小时，则只有司南暂停，云平台正常上线。

Operation：固定为 bytes: chip_protect。

推送消息格式：

- **ProtectFlag** (Bool): [不可空] 维护标志。true 为维护开始，false 维护结束。
- **DurativeTime** (Uint64_t): [不可空] 维护持续时间（单位：分钟）。
- **LastTime** (Uint64_t): [不可空] 时间戳。

示例:

```

1 {
2   "ProtectFlag":true,
3   "DurativeTime":10,
4   "LastTime":1705307288685
5 }
```

4 类型定义

4.1 任务状态类型

状态码	说明
0	未知状态
1	任务还在排队
2	任务正在运行
3	找不到对应的任务信息
4	任务计算失败
5	任务计算完成
6	任务重新发送
7	任务正在编译
8	任务编译完成

4.2 错误码类型

错误码	说明
0	无错误
1	未知错误
2	计算任务参数错误
3	Json 错误
4	任务队列满（司南发送任务超过 200 条时返回此错误）

5 指令定义

5.1 包含的基础门类型与参数说明

光量子平台的基础门类型与超导平台不同，ConvertQProg 字段保持光量子平台专用格式，不强制要求与超导一致。

支持的光量子门包括：X, Y, Z, H, T, S, X1, Y1, Z1, CR, RZ, RY, RX, U1, U2, U3, U4, TOFFOLI, CNOT, CZ, SWAP, CCNOT, MEASURE 等。